

Detailed Evaluation Report: Security Analysis and Risk Scoring

1. Scope

This report explains how evaluation works in the current implementation, with emphasis on:

- Security analysis behavior and tool integration.
- Risk scoring logic and decision policy.
- Exact information flow into the risk scoring engine (what data it uses and where it comes from).

Primary implementation references:

- `Eval/evaluate_generated_code.py`
 - `Eval/securityAnalysis.py`
 - `Eval/riskScoring.py`
 - `Eval/quickVal.py`
-

2. End-to-End Evaluation Pipeline

The evaluation entrypoint is `evaluate_code_text` in `Eval/evaluate_generated_code.py`.

High-level sequence:

1. Quick heuristic validation (`QuickVal.validate`) is executed on the code string.
2. Security tool analysis (`Security.analyze`) is executed, usually using the real file path.
3. Findings from security analysis are passed to `RiskScorer.score`.
4. A unified report is assembled, including:
 - quality/heuristic metrics,
 - security analysis outputs,
 - risk score, risk level, risk action,
 - detailed per-finding risk breakdown.

Important separation:

- Quick validation contributes to `heuristic_risk_score` and can affect approval logic.
 - Multi-factor risk scoring uses `security_report["findings"]` (not `QuickVal` issues directly).
-

3. Security Analysis: Implementation Details

Security analysis is implemented in `Eval/securityAnalysis.py` via class `Security`.

3.1 Inputs

- `code` string in constructor.
- Optional `file_path` in `analyze(file_path=...)`.

If no file path is provided, all external scanners are skipped and tool statuses are set to skipped.

3.2 Output Schema

`analyze(...)` returns:

- `issues`: list of significant findings and scanner/runtime failures.
- `warnings`: list of low-severity findings or non-blocking tool problems.
- `findings`: normalized structured findings for risk scoring.
- `tool_status`: status for `bandit`, `semgrep`, and `owasp`.

3.3 Scanner Integration

A) Bandit

Execution:

- Command: `bandit -r <file_path> -f json`
- Return codes 0 and 1 are treated as normal (1 commonly means findings exist).

Behavior:

- Parses JSON `results`.
- Builds finding records with source, severity, message, metadata fields such as test id and line number.
- Routes MEDIUM/HIGH to issues, LOW to warnings.
- If tool is missing, adds warning and skips.
- If command fails (unexpected return code), adds issue.

B) Semgrep

Execution:

- Command: `semgrep --config p/ci <file_path> --json`
- Return codes 0 and 1 are treated as normal.

Behavior:

- Parses JSON `results`.
- Uses `extra.severity` and `extra.message` to construct finding records.
- Routes ERROR/WARNING to issues, INFO-style values to warnings.
- Missing tool produces warning.
- Parse/command failures produce issues.

C) OWASP Dependency-Check

Execution prerequisites:

- Dependency-Check executable must exist (PATH or local bundled path).
- Java must be available (JAVA_HOME or PATH).

Target selection:

- Scanner first attempts to find a dependency manifest near the target file: `requirements.txt`, `Pipfile.lock`, `poetry.lock`, `pyproject.toml`, `setup.py`, Node/Java/Go/Rust/Ruby lock files.
- If no manifest is found, scanner falls back to scanning the file path itself.

Command shape:

- `dependency-check --project SecurityAnalysis --scan <target> --format JSON --out <temp_dir> --noupdate`

Behavior:

- Parses `dependency-check-report.json` from temporary output directory.
- For each vulnerability in each dependency, creates finding records with source, severity, message, vuln id, dependency file name.
- Routes CRITICAL/HIGH/MEDIUM to issues; lower severities to warnings.
- Handles timeout, parse errors, missing report, and execution errors.

3.4 Finding Record Normalization for Scoring

Each scanner emits dictionaries into `findings` with common core fields:

- `source`
- `severity`
- `message`

Additional metadata may be present (line number, dependency, check id, etc.).

Risk scoring only requires source/severity/message; extra metadata is preserved for traceability.

4. Risk Scoring: Model and Runtime Behavior

Risk scoring is implemented in `Eval/riskScoring.py` via class `RiskScorer`.

4.1 Input Contract

`RiskScorer.score` takes:

- `code`: full code string under evaluation.
- `findings`: iterable of finding dictionaries (typically from security analysis).

Optional constructor context:

- `deployment_context` defaults to `internal`.
- Supported context behavior is explicitly implemented for: `public`, `internal`, `onprem`, `sandbox`.

4.2 Core Formula

Per finding:

Risk Score = 100 x (0.35 x Severity + 0.25 x Exploitability + 0.25 x Exposure + 0.15 x Confidence)

Component values are in [0, 1].

4.3 Severity Processing

Severity text is normalized to lowercase and mapped:

- critical: 10
- high: 8
- medium: 5
- warning: 3
- low: 2
- info: 1
- unknown values default to low

Severity component = mapped value / 10.

4.4 Exploitability Processing

Exploitability is computed using message plus code content:

1. If high-risk keywords appear (for example, remote code execution, command injection, eval, exec, subprocess), exploitability = 1.0.
2. Else if medium-risk keywords appear (for example, xss, csrf, missing auth), exploitability = 0.8.
3. Else if low-risk keywords appear (warning/informational style), exploitability = 0.2.
4. Else apply source-specific fallback by severity:
 - Bandit and Semgrep have their own severity-to-exploitability mappings.
 - QuickVal is fixed at 0.4.
 - Unknown source defaults to 0.5.

4.5 Exposure Processing

Exposure uses deployment context with code-based inference:

- `public` => fixed 1.0.
- `internal` => max(0.6, inferred_from_code).
- `onprem` => min(inferred_from_code, 0.3).
- `sandbox` => min(inferred_from_code, 0.1).
- other contexts => inferred_from_code only.

Code inference looks for patterns:

- Public-facing indicators: route decorators, FastAPI/Flask/server/listen/request patterns => 1.0.
- Internal cloud/service indicators: aws/boto3/lambda, etc. => 0.6.
- Sandbox/test indicators: pytest/unittest/sandbox => 0.1.
- Main guard fallback => 0.6.
- Default => 0.6.

4.6 Confidence Processing

Confidence measures reliability and corroboration:

- 1.0 when the same normalized finding text appears from at least two sources.
- 0.8 for high/critical findings from Bandit or Semgrep.

- 0.3 for QuickVal source.
- 0.8 for other high/critical findings.
- 0.5 for medium findings.
- 0.4 for low/warning/info findings.

Cross-tool corroboration depends on message normalization:

- lowercase
- replace hex numbers with
- replace numerics with
- remove noisy symbols
- collapse whitespace
- truncate to 240 chars

4.7 Aggregation and Decision

- Each finding receives its own risk score and classification.
- Overall risk score is the maximum per-finding score (conservative gate policy).
- Risk levels and actions:
 - = 80: Critical => Block deployment
 - = 60: High => Require manual approval
 - = 40: Medium => Allow with warning
 - < 40: Low => Log for monitoring

The final risk report includes:

- `risk_score`, `risk_level`, `recommended_action`
- `risk_summary` (counts by level and source)
- `risk_findings` (full per-finding breakdown)

5. How Risk Scoring Takes Information (Data Lineage)

This section directly answers how information is consumed by risk scoring.

5.1 Data Sources

- Code source text from the evaluated file.
- Structured scanner findings from `securityAnalysis.Security.analyze`.
- Deployment context argument from evaluator CLI or API (default internal).

5.2 Field-Level Ingestion

For each finding passed to `RiskScorer.score`:

- `severity` feeds severity normalization and severity component.
- `message` feeds:
 - exploitability keyword detection,

- normalized grouping key for confidence corroboration.
- **source** feeds:
 - exploitability fallback mapping,
 - confidence logic rules.

From the **code** text:

- contributes to exploitability (keywords in code text are considered),
- contributes to exposure inference (API/server/internal/sandbox indicators).

From **deployment_context**:

- directly bounds/overrides inferred exposure.

5.3 Practical Flow Example

1. Semgrep reports WARNING with message containing command injection.
2. Risk scorer normalizes severity (warning => value 3 => severity score 0.3).
3. Exploitability sees command injection keyword => 1.0.
4. Exposure is derived from context and code (for public context => 1.0).
5. Confidence is computed from source/severity and cross-tool matching.
6. Weighted score is produced and classified.
7. Overall gate uses max score among all findings.

6. Relationship with Quick Validation

QuickVal performs a lightweight AST and string-based safety check:

- forbidden imports,
- forbidden function calls,
- syntax and loop complexity heuristics,
- optional model self-review signal.

Its outputs affect:

- **heuristic_risk_score**,
- approval state in evaluator logic,
- issue notes in final report.

However, the multi-factor risk score itself is computed from security findings plus code/context, not from QuickVal issues directly.

7. Strengths, Constraints, and Notes

Strengths:

- Multi-tool security ingestion with structured findings.
- Context-aware exposure handling for public/internal/onprem/sandbox.
- Conservative max-score decision model suitable for deployment gates.

- Confidence boosting when findings are corroborated by multiple sources.

Constraints and caveats:

- Risk scoring quality depends on scanner availability and parse quality.
 - Exposure and exploitability are heuristic pattern-based estimates.
 - Message normalization may merge distinct findings if text becomes too similar.
 - Dependency-Check requires Java and the scanner binary.
 - There is an internal identifier typo (`SANCDBOX_PATTERNS`) that does not break behavior but should be cleaned up for readability.
-

8. Suggested Improvements

1. Add schema validation for incoming finding records before scoring.
 2. Externalize weights and thresholds to configuration for experiment reproducibility.
 3. Add calibration tests linking known CVE examples to expected risk bands.
 4. Incorporate CVSS fields directly when available from OWASP Dependency-Check.
 5. Add unit tests for edge cases in normalization and cross-tool confidence grouping.
 6. Include tool execution metadata (duration, skipped reason) in final policy logs.
-

9. Conclusion

The current evaluation framework combines heuristic checks and tool-based security analysis, then applies a deterministic multi-factor risk model for policy decisions. Risk scoring consumes three primary information channels: scanner findings, code content, and deployment context. This design enables context-sensitive and automation-friendly security gating in the evaluation pipeline.